Port Scanning with OS & Version Detection

A from-scratch Nmap: Recon → Exploitation → Defense

Project Showcase & Presentation Outline

CSE 406 — Computer Security Sessional

Department of CSE, BUET

The whole pipeline at a glance

A single attacker toolkit that chains five stages, plus a defender that watches the same wire.

Stage	What it produces
1. Port scanning	open / closed / filtered ports (raw SYN)
2. Banner grabbing	*what* service + version runs on each port
3. OS fingerprinting	probabilistic OS family + version tier
4. Attacking	OS-aware, chained exploits with proof-of-access
5. Defending	multi-vector scan detector + auto-mitigation

The one-line pitch
Every frame, checksum and heuristic is hand-built — no Nmap, no Scapy. Npcap is used only as raw Layer-2 packet I/O on Windows.

Stage 1 — Port Scanning

Stage 1 — TCP SYN Port Scanning

Goal: find open ports without completing the handshake (half-open scan).

How it works

- Hand-craft **Ethernet + IPv4 + TCP** → frames byte-by-byte, incl. the Internet checksum.
- ARP-resolve the target MAC; send one SYN per port.
- Classify the reply:
  - **SYN/ACK** = OPEN
  - **RST** = CLOSED
  - no reply = FILTERED / no-response

Key talking points

- Raw send/recv via **Npcap** ctypes bindings — not sockets.
- First SYN/ACK (or RST) is captured as the *fingerprint candidate*.
- Safety bounds: private IPv4 only, ≤ 50 ports/run, min 0.2 s delay, one SYN per port, no IP-range sweep.

Files: `common/packet.py`, `common/npcap.py`, `attacker/scanner.py`

Stage 2 — Banner Grabbing

Stage 2 — Service Banner Grabbing

Goal: identify *what* runs on each open port and its version.

How it works

- Uses *full* TCP sockets — a complete handshake is needed to talk to the service (no stealth gain from raw frames here).
- **Passive** services (FTP, SSH, SMTP, POP3…) announce a banner on connect — just read it.
- **Active** services (HTTP) need a nudge: send `HEAD / HTTP/1.0` and parse the `Server:` header.

File: `attacker/banner.py`

Key talking points

- Port → protocol hint table (21=ftp, 22=ssh, 80=http…).
- Regex-parses version strings, e.g. `vsftpd 3.0.5`, `OpenSSH_8.2p1`, `Microsoft-IIS/10.0`.
- Banners are fed forward as cross-layer evidence for fingerprinting (Stage 3) and payload choice (Stage 4).

Stage 3 — OS Fingerprinting

Stage 3 — OS Fingerprinting (the centrepiece)

Goal: a probability distribution over OS families with evidence — not a single black-box guess.

Passive TCP/IP features

- Initial **TTL** (32/64/128/255, hop-corrected)
- TCP **window** size
- **DF** bit; **TCP option order** (M,W,S vs M,S,T,W…)
- MSS, window-scale, SACK, timestamp presence

Active / multi-probe features

- **IP ID** sequence class (zero / incremental / random)
- **TCP timestamp** clock rate → Hz + *uptime*
- **ECN** (RFC 3168) negotiation behaviour
- **Invalid-flags** probe (SYN+FIN rule-breaker)
- **ICMP/UDP** battery (echo code, DF, quote length)
- **Cross-layer** banner correlation

Stage 3 — Why it is honest (and why that wins)

Coarse-but-honest version tiers

- Windows: *legacy 9x* / *7-8-Vista* / *10-11-Server*
- Linux: *2.4-2.6* vs *modern 3/4/5+*
- macOS / BSD; Network device
- Ties are shown as a *range*, each tier with its own %.

File: `attacker/fingerprint.py`

Talking points

- Every score carries a human-readable **reason** line.
- Confidence is damped when evidence is thin.
- Self-authored scoring, transparent — vs Nmap's 2000-entry DB.
- TCP fields alone cannot split Win7/8/10/11 (all TTL 128, DF, same option layout) — so we don't pretend to.

Stage 4 — Attacking

Stage 4 — OS-Aware Exploitation

Goal: turn recon into proof-of-access, adapting payloads to the fingerprinted OS.

Phase 4 — payload selection

- Windows → `win.ini`, `whoami`
- Linux → `/etc/passwd`, `id`
- macOS/BSD → `/etc/passwd`, `/etc/hosts`

Phase 5 — exploit battery

- FTP anonymous login (T1078)
- FTP write → web shell → RCE (chained)
- FTP bounce (PORT-command pivot)

More exploits

- SSH weak-credential attack (T1110), bounded wordlist
- HTTP directory traversal (OS-aware payloads + URL-encoded bypasses)

Safety / ethics

- Web shell runs a whitelist of read-only commands only.
- Automatic cleanup of uploaded artifacts.
- Bounded attempts, delays, private IPv4 targets only.

Files: `attacker/exploits/*.py`, `os_payloads.py`

Stage 5 — Defending

Stage 5 — Multi-Vector Scan Detection & Mitigation

Goal: the flip side — detect our own attack on the wire and shut it down.

Detection vectors

- **Fast SYN scan:** sliding window — threshold + distinct-port diversity (e.g. ≥ 8 SYN / ≥ 5 ports in 10 s).
- **Slow / stealth scan:** long-window rate check (60 s).
- **Invalid flags:** NULL, FIN-only, XMAS, SYN+FIN.
- **Canary honeyport:** any touch = instant alert.

Files: `defense/detector.py`, `defense/main.py`

Mitigation & forensics

- **Cross-platform block:** Windows `netsh` / Linux `iptables`.
- **Self-healing:** auto-release after TTL (default 300 s).
- **Whitelist** to suppress known-good sources.
- Structured JSON **incident log** (who, when, why, rule).

Recon → Fingerprint → Exploit → Defend

every byte hand-built.

If asked — one-line answers to keep ready

- **Why raw frames?** To do a true half-open SYN scan and read stack quirks sockets hide.
- **Why probabilistic OS guess?** No small TCP/IP field set uniquely IDs a modern OS — p0f / Nmap weigh features in concert too.
- **What's novel vs Nmap?** Transparent, self-authored scoring with per-feature reasons and honest version tiers.
- **Is it safe?** Private targets, bounded probes/attempts, read-only proofs, whitelist commands, auto-cleanup.
- **Attacker *and* defender?** Yes — same wire, both sides, demonstrating detection of our own tooling.